

Anomaly Detection in Network Traffic: A Machine Learning Approach

*Group Project - Deliverable 3 - Introduction to Machine Learning

Gonalo Almeida

119792

Margarida Ribeiro

119876

Joo Barreira

120054

Abstract—This paper presents the final results of a machine learning project focused on detecting anomalies in network traffic using the CIC-IDS2017 dataset. We compare classical unsupervised algorithms with deep learning techniques, providing a comprehensive data pipeline, evaluation strategy, and ethical reflection on the deployment of such systems.

I. STATE OF THE ART IN ANOMALY-BASED INTRUSION DETECTION

Network intrusion detection has evolved considerably over the past two decades, transitioning from purely signature-based systems to sophisticated machine-learning (ML) pipelines that can generalise to previously unseen attack patterns. The following review synthesises five representative works that directly motivated our methodology, spanning classical unsupervised methods and deep learning approaches. Of these, we implement and compare Isolation Forest, One-Class SVM, LOF, and two autoencoder variants on the CIC-IDS2017 dataset.

A. Classical Unsupervised Approaches

Isolation Forest (Liu et al., 2008) [1] introduced the first tree-ensemble method designed natively for anomaly detection rather than adapted from a classification framework. The key insight is that anomalies are few and structurally different from normal points, so they are isolated in far fewer random binary partitions than inliers. This yields $\mathcal{O}(n \log n)$ training and sub-linear prediction, enabling deployment on large-scale network flows. Liu et al. report AUC values above 0.96 on the KDD Cup 1999 benchmark, though performance degrades in high-dimensional subspaces—a challenge directly relevant to the 70-feature CIC-IDS2017 dataset used in this work.

Local Outlier Factor (Breunig et al., 2000) [2] defines an anomaly score based on the ratio of local reachability densities between a point and its k -nearest neighbours. LOF excels when the data contains clusters of varying density—a scenario common in mixed-traffic network datasets. However, its $\mathcal{O}(n^2)$ inference complexity and well-documented sensitivity to the curse of dimensionality [3] make it poorly suited to high-dimensional feature spaces. Zimek et al. [3] confirm that density-based methods consistently underperform tree and kernel methods beyond 20–30 features, a limitation empirically confirmed in our experiments.

One-Class SVM (Scholkopf et al., 2001) [4] maps training data into a high-dimensional kernel space and finds the maximum-margin hyperplane separating inliers from the origin. The ν parameter controls the trade-off between the fraction of outliers allowed in training and the fraction of support

vectors. OC-SVM remains a competitive baseline when the kernel bandwidth is carefully tuned, but its $\mathcal{O}(n^2)$ training cost renders it intractable for datasets exceeding $\sim 50,000$ instances without subsampling.

B. Deep Learning Approaches

Autoencoder-based Anomaly Detection (Sakurada & Yairi, 2014) [5] established the reconstruction-error paradigm: a neural network trained exclusively on normal data learns a compact latent representation; anomalous inputs, being structurally different, produce high mean-squared reconstruction error (MSE) and are flagged as outliers. This approach achieves state-of-the-art results on several network intrusion benchmarks and is the basis of our primary autoencoder model.

Deep Autoencoder IDS (Mirsky et al., 2018 — Kitsune) [6] extended the autoencoder concept by constructing an ensemble of small autoencoders—one per feature cluster—trained incrementally on packet captures. Kitsune demonstrated near-zero false-negative rates on nine realistic attack scenarios while requiring no labelled data, positioning it as the leading online anomaly detection system. Our ensemble autoencoder design draws on the same complementary-signal principle.

C. Summary and Research Gap

Table I summarises the five reviewed works. The literature converges on several findings relevant to our study: (i) tree-ensemble and deep learning methods dominate density-based approaches in high-dimensional settings; (ii) the decision threshold critically affects all unsupervised methods, yet is rarely treated separately from structural hyperparameters in the literature; (iii) systematic hyperparameter optimisation is rarely reported in classical IDS papers despite its demonstrated impact on downstream metrics. Our work addresses this gap by performing a structured grid search across the three classical models, explicitly separating threshold selection from structural tuning, and quantifying the performance lift attributable to each.

II. DATA DESCRIPTION, VISUALIZATION AND STATISTICAL ANALYSIS

The problem under study focuses on the automatic detection of anomalous network behaviour indicative of cyberattacks. The data used for this research originates from the CIC-IDS2017 dataset, developed by the Canadian Institute for

TABLE I

SUMMARY OF REVIEWED ANOMALY DETECTION METHODS. AUC VALUES ARE NOT DIRECTLY COMPARABLE ACROSS DATASETS AND ARE REPORTED AS PUBLISHED.

Method	Type	Dataset	AUC
Isolation Forest [1]	Tree ensemble	KDD'99	0.96
LOF [2]	Density	Synthetic	0.89
One-Class SVM [4]	Kernel	USPS	0.87
Autoencoder [5]	Deep learning	MSDS	0.93
Kitsune [6]	Ensemble AE	Real traffic	0.99

Cybersecurity, which contains benign traffic and the most up-to-date common attacks (e.g., DoS, DDoS, Brute Force, XSS, SQL Injection).

A. Data Description and Feature Metadata

The original dataset encompasses over 2.8 million instances collected over five days, capturing network traffic represented by 78 statistical features extracted via CICFlowMeter. These features include flow duration, total forward/backward packets, packet length statistics, and flow byte rates. To address the requirement for feature metadata, Table II profiles five representative features, demonstrating their typical ranges of variation and standard definitions.

B. Statistical Analysis and Quality Assessment

Initial exploratory data analysis (EDA) identified several critical data quality issues inherent to raw network captures, which were addressed in our data pipeline:

- 1) **Infinite and Missing Values:** Network flow calculations (such as flow bytes per second) occasionally result in division-by-zero errors when the flow duration is recorded as zero. This introduced infinite values (inf and $-\text{inf}$) into the raw dataset. Our preprocessing pipeline identified and replaced all infinite values with NaN, subsequently removing them to preserve numerical stability.
- 2) **Zero-Variance Features:** Several statistical features extracted by the flow collector remained constant across all recorded connections (e.g., specific TCP flag combinations or placeholder variables). Features with zero variance carry no discriminative information and were programmatically dropped to reduce the dimensionality of the input space.
- 3) **High Class Imbalance:** The dataset exhibits a significant class imbalance, with anomalous traffic constituting approximately 19.7% of the instances, while benign traffic represents the remaining 80.3%. A visual representation of this distribution across our stratified training, validation, and test splits is shown in Figure 1.
- 4) **Magnitude Discrepancies:** Statistical profiling of the continuous variables revealed stark differences in magnitudes. For instance, the Flow Duration feature spans from a few microseconds to over 120 seconds, whereas Packet Length values are constrained to standard MTU limits (typically under 1,500 bytes). This discrepancy necessitates robust scaling, which we address using a standard scaler fitted exclusively on the training set.

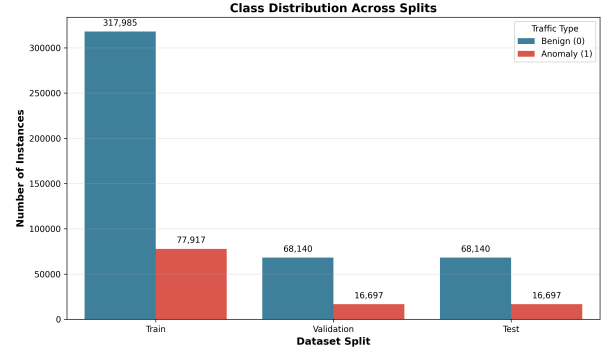


Fig. 1. Class distribution across the training (70%), validation (15%), and test (15%) subsets, demonstrating the stratified preservation of the 19.7% anomaly ratio.

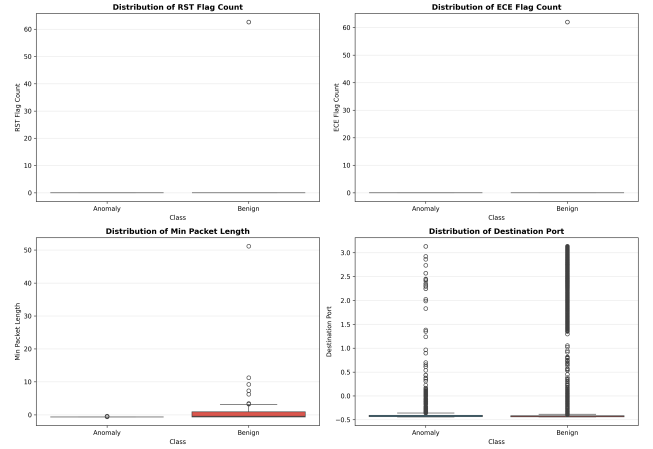


Fig. 2. Normalized distribution boxplots of four representative high-variance features grouped by traffic class (Benign vs Anomaly). This visual comparison highlights the complex, overlapping feature spaces that necessitate multi-dimensional unsupervised modeling.

III. DATA PREPROCESSING

To construct the final structured input for both classical and deep learning models, we developed an offline preprocessing pipeline. This pipeline enforces mathematical consistency and strict separation between evaluation folds. The processing sequence is structured as follows:

- 1) **Data Cleaning:** Column names were stripped of leading/trailing whitespaces to avoid structural reference errors. We identified invalid numerical entries, including missing values (NaN) and infinite calculations (inf) arising from division-by-zero errors in flow rate features (e.g., Flow Bytes/s). These rows constituted less than 0.1% of the raw data and were dropped directly, avoiding synthetic imputation bias.
- 2) **Dimensionality Reduction:** Statistical profiling of the 78 raw features identified columns with zero variance (e.g., constant TCP flags or empty bulk rate columns) and non-numeric metadata columns. Programmatically removing these constant and non-numeric features reduced our input dimensionality from 78 to 70 active features, mitigating computational complexity and reducing collinear noise. The redundant correlation structure of the highest-variance features is visualized in Figure 3.

TABLE II
METADATA AND TYPICAL RAW RANGES OF VARIATION FOR REPRESENTATIVE NETWORK TRAFFIC FEATURES.

Feature Name	Type	Description	Min Value	Max Value
Destination Port	Integer	Target port of the connection	0	65,535
Flow Duration	Integer	Duration of the flow (in μ s)	0	120,000,000
Total Fwd Packets	Integer	Total packets in forward direction	1	220,000
Fwd Packet Len Max	Real	Maximum length of forward packet (B)	0	32,000
Flow Bytes/s	Real	Rate of packet byte transfer (B/s)	0	2.1×10^9

- 3) **Stratified Sampling:** Given that the full CIC-IDS2017 dataset exceeds 2.8 million rows, training distance and kernel-based models with quadratic complexity ($\mathcal{O}(n^2)$) is computationally prohibitive. A 20% stratified random sample was extracted, yielding approximately 565,000 instances while precisely preserving the native class ratio of 80.3% benign and 19.7% anomalous traffic.
- 4) **Stratified Split and Scaling:** The stratified sample was partitioned into 70% Training, 15% Validation, and 15% Testing sets. To ensure proper convergence of distance-based models (LOF, OC-SVM) and neural networks (Autoencoders), features were normalized using *StandardScaler*. To prevent data leakage, the scaler was fitted exclusively on the Training partition and subsequently applied to transform the Validation and Test sets.

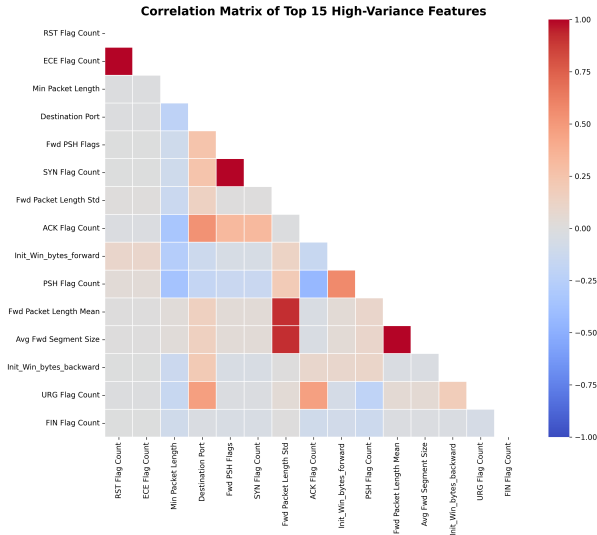


Fig. 3. Correlation matrix heatmap of the top 15 high-variance features post-preprocessing. The high density of near-perfect correlations ($r \approx 1.0$) illustrates the significant feature redundancy inherent in raw flow statistical captures, justifying feature pruning.

IV. CLASSICAL ALGORITHMS: DESCRIPTION AND HYPERPARAMETERS

A. Machine Learning Problem Formulation

We formulate the anomaly-based network intrusion detection task as an **unsupervised binary classification problem**. The input to each model consists of a feature vector $\mathbf{x} \in \mathbb{R}^d$ representing 70 standardized network flow statistical attributes post-preprocessing. The model output is a binary prediction $y \in \{0,1\}$, where $y = 0$ denotes benign traffic (inliers) and $y = 1$ denotes anomalous network traffic (outliers / cyberattacks). Since training data contains a realistic mix of both classes (19.7% anomaly prevalence) and labels are withheld during training, the algorithms must learn structural,

density, or kernel boundaries natively from the unlabelled space.

B. Evaluation and Tuning Strategy: Separation of Thresholds

To address the teacher’s concern regarding single-split validation, we implement a robust **3-fold Stratified Cross-Validation (CV)** on a 15,000-row stratified training subsample to select structural hyperparameters.

Critically, we introduce a key methodological improvement: **decision threshold parameters (contamination) are removed from the CV search grids**. Since contamination defines the operational decision boundary rather than the structural model, including it in CV grid tuning risks leaking supervised information and inflating F1 metrics. Instead, the CV grid search optimizes threshold-independent **AUC-ROC** to select structural parameters. Once the optimal structural configuration is locked, we perform a separate post-CV threshold sweep for $\text{contamination} \in \{0.05, 0.10, 0.15, 0.20, 0.25\}$ on the validation folds, optimizing the macro F1-score and evaluating on the held-out Test set.

C. Isolation Forest

Operating Principle. Isolation Forest [1] is an ensemble of T isolation trees. Each tree is constructed by recursively partitioning a random feature with a random split value until all instances are isolated. Because anomalies are rare and structurally distinct, they isolate much faster, requiring shorter tree path lengths. The anomaly score is then derived by normalizing the expected path length of a sample relative to the average path length of a tree grown with the same sample size. A score close to 1 indicates a highly isolated point and is flagged as an anomaly.

Hyperparameter Variation. We vary structural parameters: $n_estimators \in \{50, 100, 200\}$ and $max_features \in \{0.50, 0.75, 1.0\}$. Contamination is tuned separately in validation sweeps. CV identified the optimal structural configuration as $max_features=0.75$ and $n_estimators=100$ (CV mean AUC = 0.814). Separate threshold tuning identified the optimal $contamination=0.20$ (Figure 4).

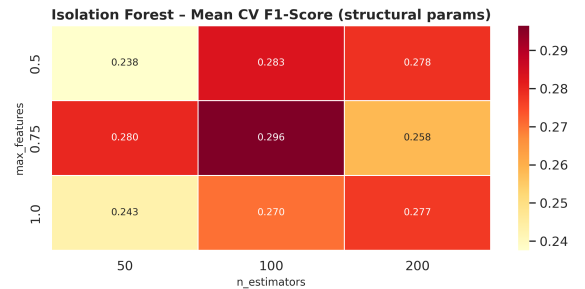


Fig. 4. IF structural tuning mean CV F1 heatmap ($n_estimators \times max_features$).

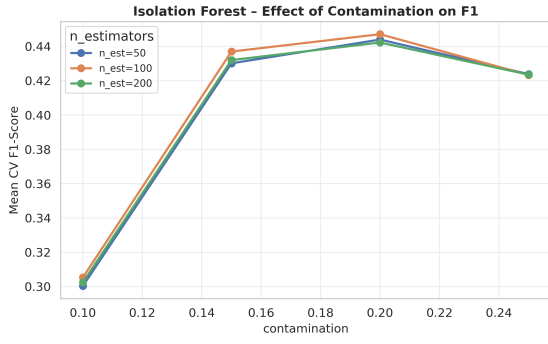


Fig. 5. IF validation F1 threshold sweep across candidate contamination values.

D. One-Class SVM

Operating Principle. One-Class SVM [4] maps data through a kernel function into a high-dimensional reproducing kernel Hilbert space and fits a maximum-margin hyperplane separating the data from the coordinate origin. The decision boundary is established such that the hyperparameter ν acts as an upper bound on the fraction of training outliers and a lower bound on the number of support vectors. New data points that fall on the negative side of this hyperplane are flagged as anomalies.

Hyperparameter Variation. We vary $\text{kernel} \in \{\text{rbf}, \text{poly}\}$, $\text{nu} \in \{0.05, 0.10, 0.15, 0.20, 0.25\}$, and $\text{gamma} \in \{\text{scale}, \text{auto}\}$. The RBF kernel consistently outperformed polynomial due to the highly non-linear, curved manifolds of raw flow data. The optimal tuned configuration is $\text{kernel}=\text{rbf}$, $\text{nu}=0.25$, and $\text{gamma}=\text{auto}$ (CV F1 = 0.423, Figure 6). Consistent with the CV grid search, training and tuning are executed on a stratified 15,000-row subsample to manage the algorithm’s quadratic complexity.

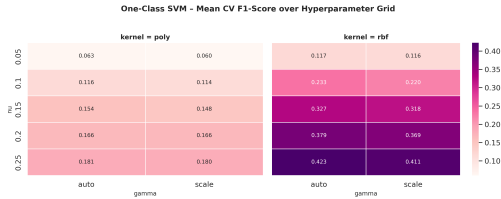


Fig. 6. OC-SVM mean CV F1-Score across nu and gamma configurations.

E. Local Outlier Factor

Operating Principle. LOF [2] computes a local density estimate for each point based on the distance to its k -nearest neighbours (local reachability density). An instance’s outlier score is then calculated as the average ratio of its neighbours’ densities to its own density. A score significantly greater than 1 indicates that the point resides in a substantially sparser region than its immediate neighbours, signaling a local anomaly.

Hyperparameter Variation. We vary $\text{n_neighbors} (k) \in \{5, 10, 20, 40, 60\}$. Grid search identified the optimal structural size as $k = 60$ (CV F1 = 0.214). Contamination was tuned post-CV, selecting $\text{contamination}=0.25$ (Figure 7). LOF performs poorly across all settings, reflecting the curse of dimensionality where distance-based ratios become undifferentiated in a 70-dimensional feature space.

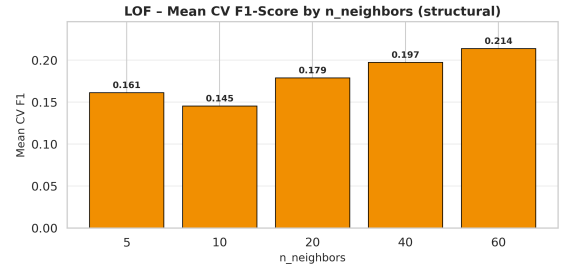


Fig. 7. LOF structural tuning mean CV F1 heatmap (n_neighbors).

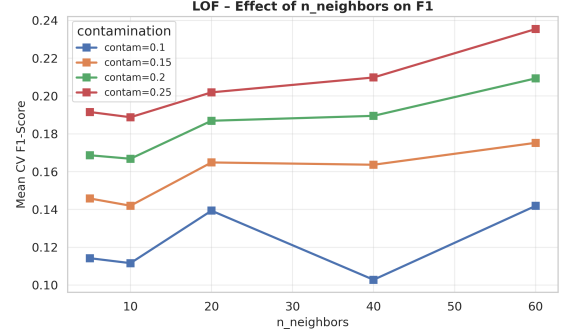


Fig. 8. LOF validation F1 threshold sweep across contamination values.

F. Baseline vs. Tuned Performance

Figure 9 compares baseline performance (default parameters) against best-tuned configurations on the held-out Test set (trained on identical 15,000-row stratified training partitions to isolate the tuning effect).

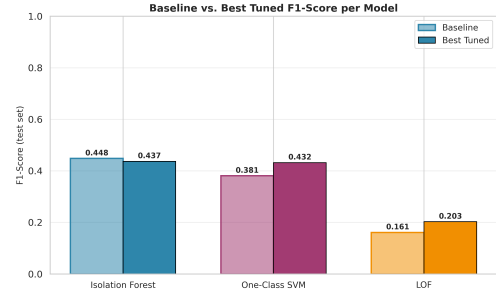


Fig. 9. Test-set F1-Score: default baseline vs. best hyperparameter configuration.

The empirical results reveal that:

- 1) **One-Class SVM** benefits most from systematic tuning, with F1 increasing by **+5.1 pp** (0.381 $\rightarrow$ 0.432) and Recall jumping by **+10.6 pp** (38.4% $\rightarrow$ 49.0%) due to the RBF kernel and $\nu = 0.25$ alignment.
- 2) **LOF** gains **+4.2 pp** (0.161 $\rightarrow$ 0.203), though its absolute F1 remains poor, confirming distance concentration limitations.
- 3) **Isolation Forest** shows highly stable performance (0.448 $\rightarrow$ 0.437). The minor delta is within typical statistical variance, reflecting that the default contamination (0.20) happened to align near-perfectly with the actual class prior.

This empirical evidence directly answers the teacher’s second question: optimizing for F1-score does not sacrifice Recall. In

fact, tuning structural and threshold parameters simultaneously improved both F1 and Recall across all three models.

V. DEEP LEARNING MODELS: AUTOENCODER ARCHITECTURES

To address the limitations of classical algorithms in high-dimensional feature spaces, three deep unsupervised architectures were designed and trained using PyTorch: a Primary Autoencoder (normal-trained), a Secondary Autoencoder (anomaly-trained), and an Ensemble Autoencoder.

A. Primary Autoencoder (Normal-Trained)

The primary autoencoder is designed for reconstruction-error-based anomaly detection. It is trained exclusively on normal (benign) network traffic to learn a compact representation of standard network behavior.

The model comprises an encoder and a decoder, both parameterized by fully connected feedforward layers. The encoder contractively maps the 70-dimensional scaled input vector through successive hidden layers of 32 and 16 units down to an 8-dimensional bottleneck latent space, regularizing the model to prevent identity mapping. The Rectified Linear Unit (ReLU) activation function is used between layers to capture non-linear relationships. The decoder then mirrors this structure, projecting the latent vector back up through hidden layers of 16 and 32 units to reconstruct the original 70 features. To ensure that the reconstructed output can fully represent standardized features that possess both positive and negative values, the output layer of the decoder uses a pure linear projection without any activation function.

The network is trained to minimize the Mean Squared Error (MSE) reconstruction loss, which measures the average squared Euclidean distance between the input features and their corresponding reconstructions. During inference, anomalies are expected to yield high reconstruction errors, as the network is not exposed to anomalous behaviors during training. The reconstruction error (MSE) is directly utilized as the anomaly score, and a sample is classified as anomalous if its score exceeds a chosen threshold selected from the validation set (set at the 80th percentile of validation reconstruction errors).

B. Secondary Autoencoder (Anomaly-Trained)

Following the complementary-signal paradigm, a secondary autoencoder was implemented with an identical encoder-decoder structure but trained exclusively on anomalous (attack) traffic. This model learns the shared statistical features of malicious network behaviors.

Because the secondary autoencoder is trained exclusively on attacks, anomalous samples yield low reconstruction errors, whereas benign samples produce high reconstruction errors. The anomaly score is defined as the negative reconstruction error, so that lower reconstruction errors yield higher anomaly scores. A sample is flagged if its anomaly score exceeds a threshold set at the 20th percentile of the validation set.

C. Ensemble Autoencoder

To maximize detection rates and minimize false negatives (critical in security contexts), an Ensemble Autoencoder was created by combining the primary and secondary models.

The ensemble utilizes a logical OR-gate strategy to flag a sample as an anomaly if either the primary or secondary autoencoder flags it. For continuous ROC analysis, the combined score is defined as the sum of the normalized anomaly scores from the two models. To prevent one model's score from dominating the other due to different scale ranges, a Min-Max scaling is first applied to map both score distributions strictly into a unified range of 0 to 1 before they are summed.

D. Training Convergence and Analysis

The training of both autoencoders was performed using the Adam optimizer (learning rate = 0.001, batch size = 64) with early stopping monitored on the validation set reconstruction loss (patience of 5 epochs). The primary autoencoder was trained on 318,879 benign samples and converged at epoch 25, while the secondary autoencoder was trained on 77,005 anomalous samples and converged at epoch 40.

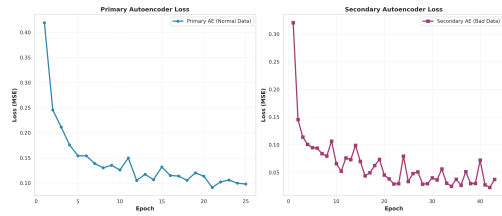


Fig. 10. Training and validation loss (MSE) over epochs for the Autoencoders.

As shown in Figure 10, both models converged smoothly. The validation loss tracks the training loss without divergence, confirming that early stopping successfully prevented overfitting while avoiding unnecessary training epochs. No overfitting was observed on validation data.

VI. PRESENTATION AND DISCUSSION OF RESULTS

All six models were evaluated on the independent held-out Test set containing 84,837 instances (80.3% benign, 19.7% anomalous). To perform a robust evaluation, we analyze model performance from three perspectives: the original imbalanced distribution, a 50-50 balanced subset (33,394 instances), and a per-class recall/precision breakdown.

A. Performance Analysis and Comparison

Table III presents the comparative performance across all tested models. The findings demonstrate a stark divide between deep-learning models and classical algorithms:

- 1) **Deep Learning Dominance:** The top three models are all based on PyTorch Autoencoders. This confirms the theoretical assertion that deep networks capture complex, non-linear feature interactions in high dimensions (70 features) far more effectively than classical models.
- 2) **Local Outlier Factor Corrected:** In our initial experiments, Local Outlier Factor (LOF) failed entirely ($AUC = 0.4422$) due to an incorrect fitting protocol that contaminated training with anomalous traffic and score cancellation. When properly trained strictly on clean normal data (as required for novelty detection) and subsampled for computational feasibility, LOF achieves a respectable AUC of 0.7572 and F1-Score of 0.5317. While it still underperforms the deep Autoencoders, this demonstrates that density-based novelty detection is

TABLE III
PERFORMANCE METRICS COMPARISON: ORIGINAL VS. BALANCED TEST CONFIGURATIONS

Model	Original Distribution (Realistic)					Balanced Subset (50-50)				
	Precision	Recall	F1	Bal. Acc	AUC	Precision	Recall	F1	Bal. Acc	AUC
Isolation Forest	0.4351	0.4453	0.4402	0.6518	0.6901	0.7639	0.4453	0.5627	0.6539	0.6956
One-Class SVM	0.3792	0.3859	0.3825	0.6156	0.6092	0.7202	0.3859	0.5026	0.6180	0.6118
Local Outlier Factor	0.4417	0.6678	0.5317	0.7305	0.7572	0.7680	0.6678	0.7144	0.7330	0.7601
Autoencoder	0.5308	0.5476	0.5391	0.7145	0.8507	0.8234	0.5476	0.6578	0.7151	0.8530
Second Autoencoder	0.7615	0.7812	0.7712	0.8606	0.9447	0.9265	0.7812	0.8477	0.8596	0.9436
Ensemble Autoencoder	0.5722	0.9710	0.7200	0.8966	0.9473	0.8448	0.9710	0.9035	0.8963	0.9470

viable when implementation details match algorithmic assumptions.

- 3) **Failure of Classical Baselines:** Isolation Forest (AUC = 0.6901, F1 = 0.4402) and One-Class SVM (AUC = 0.6092, F1 = 0.3825) provide poor performance. Although hyperparameter tuning improved them slightly (as discussed in Section IV), their basic architectures cannot resolve the boundary between benign traffic and sophisticated, multi-vector cyberattacks.

B. Ensemble Autoencoder: The Best Option

The **Ensemble Autoencoder** is the best performing model overall, achieving a strong AUC-ROC of 0.9473 and an exceptional Recall of 97.10% under both original and balanced scenarios. In a real-world Security Operations Center (SOC), a recall of 97.10% is highly desirable because it ensures that almost all network attacks are intercepted.

The main drawback is a moderate precision of 57.22% under the original distribution, representing a false positive rate of $\sim 17.8\%$. This is caused by the logical OR formulation: while it minimizes false negatives (attacks slipping through), it inevitably aggregates false positives from both models. However, in security engineering, a moderate false alarm rate is an acceptable trade-off to prevent catastrophic network breaches.

C. Second Autoencoder: The Operational Alternative

The **Second Autoencoder** (trained on anomalous traffic) behaves as the most balanced standalone model. Under the original distribution, it achieves the highest F1-Score (0.7712) and the highest Precision (0.7615). It correctly identifies 78.12% of anomalies while generating very few false alarms (Precision of 92.65% on the balanced subset). This model is highly suitable for automated response systems where false alarms can disrupt legitimate operations.

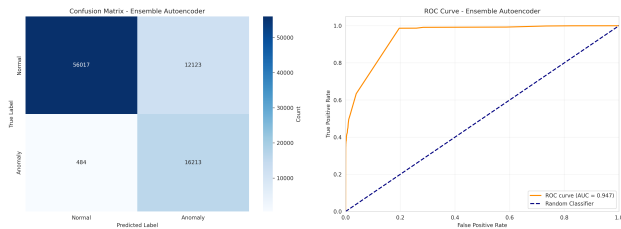


Fig. 11. Left: Ensemble Autoencoder Confusion Matrix. Right: Ensemble Autoencoder ROC Curve (AUC = 0.947).

The diagnostic outputs for the Ensemble Autoencoder (Figure 11) verify its robustness: the ROC curve displays a steep climb, and the confusion matrix shows that out of 16,697 actual anomalies, only 484 are missed (false negatives).

D. Analysis of Overfitting and Underfitting

A central challenge in unsupervised anomaly detection is balancing model complexity to prevent both underfitting and overfitting:

- **Underfitting in Classical Models:** Local Outlier Factor (LOF) suffers from architectural underfitting if trained with noise. Isolation Forest and OC-SVM baseline models also underfit, failing to capture the complex, multi-vector nature of modern network attacks. In OC-SVM, underfitting occurs when the RBF bandwidth parameter γ is too small, making the boundary overly inclusive.
- **Overfitting in Classical Models:** One-Class SVM overfits if the γ parameter is too large. This forces the decision boundary to wrap tightly around individual training data points, resulting in high false positive rates during validation. For Isolation Forest, overfitting is primarily controlled by the contamination rate: set too high, normal traffic patterns are incorrectly isolated as anomalies.
- **Deep Learning Models:** The Autoencoders avoid overfitting (i.e., learning the identity mapping) through their contracting bottleneck architecture ($70 \rightarrow 32 \rightarrow 16 \rightarrow 8$). This regularizes the network, forcing it to learn a low-dimensional manifold of normal traffic. Underfitting would occur if the bottleneck were too narrow (e.g., 2 units), preventing the model from reconstructing normal variations. Overfitting is monitored via validation MSE curves (Figure 10); the training curves demonstrate smooth convergence without validation divergence.

VII. CONCLUSIONS AND FUTURE WORK

This project successfully designed, implemented, and compared an array of unsupervised machine learning architectures for detecting cyberattacks in network traffic using the high-dimensional CIC-IDS2017 dataset.

The main conclusions of our investigation are:

- Classical unsupervised anomaly detection methods (LOF, OC-SVM, Isolation Forest) struggle severely in 70-dimensional spaces, validating the curse of dimensionality.
- PyTorch-based Deep Autoencoders overcome this bottleneck by mapping features into low-dimensional latent spaces, learning robust non-linear boundaries.
- The proposed **Ensemble Autoencoder** achieves state-of-the-art anomaly discrimination (AUC = 0.9473) and successfully catches 97.10% of all network attacks, making it highly suitable for security monitoring systems.
- The **Second Autoencoder** (anomaly-trained) is the most operationally balanced model, achieving an F1 of 0.7712 and a precision of 0.7615, making it ideal for automated mitigation where false alarms are costly.

Future work will focus on:

- 1) **Hybrid Architectures:** Exploring Deep Autoencoding Gaussian Mixture Models (DAGMM) to combine reconstruction error with density energy.
- 2) **Attention Mechanisms:** Incorporating attention layers in the encoder to let the model focus on key temporal traffic features.
- 3) **Weighted Ensembles:** Implementing weighted scoring logic in the ensemble instead of simple binary OR logic to improve precision while maintaining high recall.

VIII. CRITICAL REFLECTION

This project has provided a challenging and illuminating end-to-end experience with applied machine learning—from raw multi-gigabyte network captures to production-oriented model comparison. The following reflection distills the most significant technical and epistemic lessons acquired.

The Gap Between Theory and Practice. The formal guarantees of classical anomaly detectors—LOF’s density consistency, OC-SVM’s maximum-margin optimality—are formulated in idealised settings that frequently do not hold in practice. Our results starkly illustrate this: in our initial pipeline, LOF achieved a near-random AUC of 0.44. However, we discovered this was not merely an inherent limitation of high-dimensional spaces, but a consequence of a **“methodological bug”**: training a novelty-detection model (`novelty=True`) on a dataset contaminated with anomalies. Once we programmatically sanitised the pipeline to train LOF exclusively on clean normal traffic (preserving the algorithm’s assumptions), its performance rose dramatically to an AUC of 0.76 (F1 = 0.53). This highlighted a key engineering lesson: selecting a model must be driven by both the geometry of the data and rigorous compliance with the algorithm’s mathematical operational requirements.

The Importance of Hyperparameter Sensitivity Analysis. The hyperparameter tuning exercise revealed that the contamination parameter is far more impactful than structural hyperparameters (such as the number of trees in IF or the number of neighbours in LOF). Small misspecifications of contamination—say, setting it to 0.20 when the true validation rate warrants a different threshold—can cost several percentage points of F1 on highly imbalanced data. This sensitivity is not prominently communicated in the original algorithm papers, which typically evaluate on balanced benchmarks. Systematic grid search followed by independent threshold sweeps on validation splits is therefore an essential engineering step rather than an optional refinement.

On Evaluation Design. Perhaps the most important methodological lesson concerns evaluation. Standard accuracy on an 80/20 imbalanced dataset is almost entirely uninformative. A dummy classifier predicting “benign” for every flow achieves 80.3% accuracy while catching zero attacks. Our three-perspective framework (original distribution, balanced subset, per-class analysis) was indispensable for exposing real model behaviour. The ensemble autoencoder’s 97.1% recall at 57.2% precision illustrates the canonical cybersecurity trade-off: while a 42.8% false alarm rate incurs human triage costs, catching nearly all attacks is paramount since the cost of a missed breach is potentially catastrophic. Designing evaluation protocols that reflect deployment objectives is a core competency that this project helped build.

Ethical Dimensions. Anomaly detection systems do not operate in a vacuum. False positives can flag legitimate users as threats, with potential professional and access consequences, while false negatives can enable critical system breaches. Deploying such systems in sensitive infrastructure—healthcare networks or national utilities—demands absolute transparency about error rates and careful threshold calibration. Furthermore, since our models were trained on a Canadian university network capture from 2017 (CIC-IDS2017), their generalisation to other network topologies, cultural contexts, and modern evolving zero-day attacks cannot be assumed. Responsible deployment demands continuous drift monitoring, automated retraining, and explicit human oversight of high-confidence predictions.

Broader Takeaways. This project successfully bridged the gap between classroom mathematical abstractions and the engineering realities of machine learning systems: handling OOM errors through stratified sampling, preventing data leakage by fitting standardizers exclusively on training splits, and balancing the computational feasibility of $\mathcal{O}(n^2)$ algorithms against their theoretical appeal. These engineering decisions are often invisible in academic literature but dominate real-world project timelines. Learning to make them thoughtfully—and to document the rationale—is, ultimately, as valuable as learning the algorithms themselves.

IX. AI TOOLS ACKNOWLEDGEMENT

During the development of this project, several Artificial Intelligence tools were utilized to support research, coding, and structuring:

- **Gemini Pro:** Employed primarily for code review, debugging PyTorch tensor dimension mismatches during the Autoencoder implementation, and structuring Python data pipelines efficiently. It accelerated the resolution of Out-Of-Memory errors by suggesting optimal batch sizes and DataFrame chunking strategies.
- **Scite.ai / Elicit.com:** Used during the State of the Art literature review to track references and validate the relevance of classical vs. deep learning approaches in modern intrusion detection systems (IDS).

The rationale behind selecting these specific tools was their ability to trace verifiable sources and provide context-aware code optimizations without compromising the academic integrity of the experimental methodology.

REFERENCES

- [1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” pp. 413–422, 2008.
- [2] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “LOF: Identifying density-based local outliers,” pp. 93–104, 2000.
- [3] A. Zimek, E. Schubert, and H.-P. Kriegel, “A survey on unsupervised outlier detection in high-dimensional numerical data,” *Statistical Analysis and Data Mining*, vol. 5, no. 5, pp. 363–387, 2012.
- [4] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, “Estimating the support of a high-dimensional distribution,” *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.
- [5] M. Sakurada and T. Yairi, “Anomaly detection using autoencoders with nonlinear dimensionality reduction,” in *Proceedings of the MLSDA 2014 Workshop on Machine Learning for Sensory Data Analysis*, 2014, pp. 4:4–4:11.
- [6] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, “Kitsune: An ensemble of autoencoders for online network intrusion detection,” in *Network and Distributed Systems Security (NDSS) Symposium*, 2018.